

transcodeVR

A Batch Pipeline for VR-to-Flat Video Conversion
via FFmpeg

michel-ch

2026

Abstract

transcodeVR is a small set of Windows batch scripts that drive **FFmpeg** to reproject 180° side-by-side stereoscopic video into a flat 2D rectilinear “normal-camera” view, with optional per-folder concatenation of the resulting clips. A third script provides a hardware-accelerated 1080p downscale for ordinary footage. The project intentionally has no build system, no runtime dependencies beyond **FFmpeg**, and no source code in the conventional sense: every entry point is a self-contained **.bat** file. This report describes the goal of the project, the architecture of the solution, the **FFmpeg** pipeline used to perform the reprojection, and the trade-offs that shaped the design.

Contents

1	Goal	2
1.1	Stated objectives	2
1.2	Non-goals	2
2	Architecture	2
2.1	High-level view	2
2.2	Components	2
2.3	Folder convention	2
3	The FFmpeg pipeline	4
3.1	Per-clip VR conversion	4
3.1.1	Stage 1: crop the right eye away	4
3.1.2	Stage 2: reproject hemisphere to plane	4
3.1.3	Stage 3: encode	4
3.2	Concatenation stage	5
3.3	1080p downscale pipeline	5
4	Trade-offs	6
5	Conclusion	6

1 Goal

The project addresses a personal-archival problem: turning recordings made on a 180° stereoscopic VR camera into ordinary, playable, shareable 2D video files. VR sources stored as *half-equirectangular* side-by-side (SBS) frames look distorted and unwatchable in conventional players, and even on VR-aware players they require a headset to be enjoyed.

1.1 Stated objectives

1. **Reproject** 180° SBS source video to a flat rectilinear view, with framing tuned for an arbitrary camera mount (*e.g.* a head-mounted rig looking at a subject below the eyeline).
2. **Batch** the conversion across nested folders, so an entire archive of source clips can be processed unattended.
3. **Optionally concatenate** the per-clip outputs of each scene folder into a single playable file, since many VR recordings are split into multiple sequential takes.
4. **Strip identifying metadata** from outputs so files do not carry over camera/serial/handler information.
5. Provide a **secondary downscale path** for non-VR footage that needs to be reduced to 1080p.

1.2 Non-goals

- No GUI. The user interface is the `.bat` file.
- No cross-platform support. Windows-only.
- No content management. The scripts treat folders as opaque units; they do not parse filenames, dates, or sidcar metadata.
- No incremental processing. Re-running a script re-encodes everything; checkpointing was deliberately skipped to keep the scripts trivial.

2 Architecture

2.1 High-level view

The solution is composed of three independent batch-file entry points, each a thin orchestrator around a single `FFmpeg` invocation template. The two VR scripts share an input/output convention (`vr/` → `nonevr/`) and an encoder template; the 1080p script is conventionally separate (`input/` → `output/`, `NVENC` instead of `x264`).

2.2 Components

The two VR scripts share the per-clip encoder line verbatim; the only difference between them is whether the `concat` stage runs after every folder is processed. This duplication is deliberate: keeping each script self-contained means the user can run either one without worrying about an extra flag, and editing the encoder parameters in both files is a two-edit operation.

2.3 Folder convention

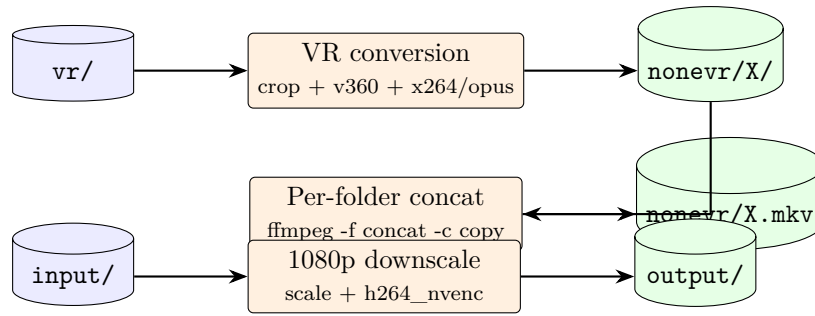


Figure 1: High-level dataflow. Both VR scripts share the `vr/` $\rightarrow$ `nonevr/` path; only `VR2Normal.bat` includes the optional per-folder concat stage. The 1080p script is fully independent.

Script	Responsibility	Output
<code>VR2Normal.bat</code>	Convert every VR clip in every subfolder of <code>vr/</code> , then concatenate the resulting clips per-folder into a single MKV.	<code>nonevr/X/*.mkv</code> , <code>nonevr/X.mkv</code>
<code>only_vr.bat</code>	Same per-clip conversion as <code>VR2Normal.bat</code> , without the concat stage. Used when each clip must remain a separate output.	<code>nonevr/X/*.mkv</code>
<code>only_1080p.bat</code>	Hardware-accelerated 1080p downscale of <code>*.MP4</code> files in <code>input/</code> . Audio stream-copied.	<code>output/X/*_1080p.MP4</code>

Table 1: The three entry-point scripts and their roles.

Listing 1: Project layout. Empty content directories ship with the repository as conventions; only `*.bat`, `LICENSE`, `README.md`, and the `docs/` tree are tracked in git.

```

transcodeVR/
  VR2Normal.bat :: VR convert + per-folder concat
  only_vr.bat  :: VR convert (no concat)
  only_1080p.bat :: 1080p downscale (NVENC)
  vr/         :: VR input (one subfolder per scene)
  nonevr/     :: VR output
  input/      :: 1080p input
  output/     :: 1080p output
  temp/       :: concat lists (auto-deleted)
  docs/       :: markdown docs
  report/     :: this report

```

The "one level of subfolders" model is the unifying convention: the input root is always a directory of *scene folders*, never of loose files. This lets the script use a simple two-level `for /d` loop and assigns clear semantics to a folder ("everything inside is one scene").

3 The FFmpeg pipeline

3.1 Per-clip VR conversion

The heart of the project is a single **FFmpeg** command applied to every input clip. It is broken across two filter stages followed by a re-encode.

Listing 2: Per-clip VR conversion command (line-broken for readability; in the source it is one line).

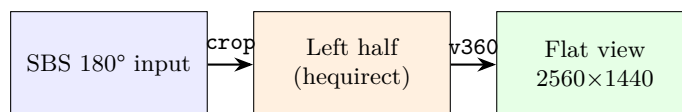
```
ffmpeg
-hide_banner -loglevel error -stats
-y
-hwaccel cuda
-i <input>
-map 0:v:0 -map 0:a
-vf "crop=w=iw/2:h=ih:x=0:y=0,
    v360=hequirect:flat:in_stereo=2d:out_stereo=2d:
        iv_fov=180:ih_fov=180:d_fov=115:
        pitch=-35:yaw=0:roll=0:
        w=2560:h=1440:interp=lanczos:reset_rot=1"
-c:v libx264 -preset medium -crf 24 -tune film
-x264-params keyint=600:bframes=7
-c:a libopus -b:a 128K
-map_metadata:g -1
-map_metadata:s:v -1
-map_metadata:s:a 0:s:a
-metadata:s "HANDLER_NAME="
<output>.mkv
```

3.1.1 Stage 1: crop the right eye away

180° SBS sources place the left eye in the left half of the frame and the right eye in the right half. Since the project produces *monoscopic* output, only one eye is needed. The `crop=iw/2:ih:0:0` step keeps the left half. Switching to the right eye is a single edit (`x=0` → `x=iw/2`).

3.1.2 Stage 2: reproject hemisphere to plane

The cropped image is then a *half-equirectangular* projection of the camera’s viewing hemisphere. FFmpeg’s `v360` filter converts it to a rectilinear view (the `flat` target):



The framing parameters control where the virtual camera looks inside the source hemisphere:

These four numbers (`d_fov`, `pitch`, `yaw`, `roll`) are the only “creative” inputs to the pipeline; all other settings are quality/encoding choices.

3.1.3 Stage 3: encode

Video is encoded with software `x264`: **preset medium**, **CRF 24**, **tune film**, **keyint 600**, **7 B-frames**. Audio is re-encoded with **libopus** at **128 kbps**. The output container is **Matroska** (`.mkv`) because it tolerates Opus and arbitrary stream layouts cleanly, and because the merge step uses `ffmpeg -f concat -c copy`, which is most reliable on MKV.

A subtle but useful detail is the metadata stripping at the end: `-map_metadata:g -1`, `-map_metadata:s:v -1`, and `-metadata:s "HANDLER_NAME="` together remove globally-scoped

Param	Value	Effect
iv_fov,ih_fov	180,180	Source covers the full hemisphere (both axes).
d_fov	115	Output diagonal field of view; smaller is “zoomed in,” larger is “wide angle.”
pitch	−35	Tilt down 35°, compensating for a camera mounted at eye level looking at a subject below.
yaw,roll	0,0	No pan, no roll.
w,h	2560,1440	1440p output.
interp	lanczos	High-quality resampling.
reset_rot	1	Discard any source-side rotation metadata.

Table 2: The v360 parameters and their visual effect.

container metadata, all per-stream video metadata, and the `HANDLER_NAME` tag that some cameras populate with serial-number-like strings. This makes outputs more anonymous and less likely to confuse downstream players.

3.2 Concatenation stage

After per-clip conversion, `VR2Normal.bat` walks each output folder, builds a concat list with `dir /b /on` (alphabetical), and runs:

```
ffmpeg -hide_banner -loglevel error -stats
-f concat -safe 0
-i <concat_list.txt>
-c copy
<merged>.mkv
```

Critically, this does *not* re-encode. Because every per-clip output was produced by the same `FFmpeg` command, all clips share identical codec parameters, and `-c copy` can stitch them without re-encoding. This makes the merge essentially a memcpy of demuxed packets, and runs at I/O speed. Folders containing zero or one file skip the concat step.

The alphabetical ordering is intentionally simple: it relies on the naming discipline of the source clips. Zero-padded numeric names (`clip01.mp4`, `clip02.mp4`, ...) sort correctly, while bare numeric names (`clip1.mp4`, `clip10.mp4`, `clip2.mp4`) do not.

3.3 1080p downscale pipeline

The 1080p script is intentionally a different shape:

```
ffmpeg -y -i <input>
-vf "scale=1920:1080"
-c:v h264_nvenc -preset p4 -cq 23
-c:a copy
<output>_1080p.MP4
```

Two notable design choices: encoding is **NVENC** (GPU) rather than x264, trading a small amount of bit-efficiency for a large speedup; and audio is **stream-copied**, since downscaling does not require touching audio. The output extension is preserved as `.MP4` (since audio is unmodified, no container change is needed).

4 Trade-offs

The project deliberately optimizes for **hackability over robustness**. Some of the consequences:

- **No idempotency**: re-running a script reprocesses every file, even if an output already exists. A `if exist skip` would be one line, but was deliberately omitted to keep the user's mental model trivial: *nuke the output folder if you want to start over*.
- **No exit code**: each script ends in `pause`, with no propagation of `errorlevel` to the caller. The scripts are intended for interactive use, not pipeline automation.
- **Hardcoded paths**: `FFMPEG_PATH` is baked into the VR scripts, and all input/output roots are absolute paths rooted at `C:\Users\%USERNAME%\Desktop\transcodeVR`. Moving the project requires editing the scripts.
- **Mixed FFmpeg invocation**: the VR scripts use a configurable `FFMPEG_PATH`; the 1080p script calls `ffmpeg` from `PATH`. This is technical debt rather than a design choice.
- **Inconsistent folder conventions** across the VR and 1080p scripts (`vr//nonevr/` vs. `input//output/`). Each pipeline is internally consistent, but they cannot share a folder.
- **Silent quirks** (*e.g.* the unused `!THREADS!` variable in `only_1080p.bat`, the misleading `"_MERGED.mkv"` string in `VR2Normal.bat`'s success message) are documented in `docs/troubleshooting`, rather than fixed, on the principle that the scripts are working code in production use and edits must be "surgical."

5 Conclusion

`transcodeVR` is, in the end, a deliberately small project: 250 lines of batch script plus an `FFmpeg` filter chain. The architectural decisions in this report (one-level folder convention, duplicated encoder template across two scripts, no idempotency, no exit codes) are all chosen to keep the system trivial to read and edit. The complexity that does exist lives almost entirely in the `FFmpeg` command line: the `v360` reprojection, the metadata-stripping flags, the `concat-list` construction. Everything else is a thin loop around it.

The full markdown documentation lives at `docs/` in the project root and is the authoritative source for usage; this report is a high-level companion that describes *why* the system has the shape it does.

Project repository: <https://github.com/michel-ch/transcodeVR>